

Artificial Neural Networks

Vid Koncilja
2025/26 , FRI, UL
vk31428@student.uni-lj.si

I. INTRODUCTION

In this report, we describe the implementation of artificial neural networks (ANNs) starting from a from-scratch NumPy implementation with manually derived backpropagation, and later extending it using PyTorch. Finally, we apply the developed models to classify FTIR spectral data of human tissue into six classes.

II. PART 1

In this part, we implemented a fully-connected multi-layer artificial neural network for classification using only NumPy. The network supports an arbitrary number of hidden layers, each with biases and sigmoid activation functions. Training is performed using gradient descent based on analytically derived gradients computed via manual backpropagation.

To verify the correctness of the implemented backpropagation algorithm, we performed gradient checking by comparing analytically computed gradients with numerical approximations obtained via finite differences. The numerical gradients were computed using a central difference scheme with a small perturbation $\epsilon = 10^{-5}$. The verification was conducted on multiple datasets, including two real datasets (Doughnut and Squares) and two synthetic datasets (Gaussian blobs and a circular decision boundary). For each dataset, several neural network architectures with different hidden layer sizes were tested.

For all tested configurations, we computed the relative error between analytical and numerical gradients. The results show that the maximum relative error is consistently on the order of 10^{-11} to 10^{-10} across all datasets and architectures.

These results confirm that the implemented backpropagation algorithm correctly computes analytical gradients and is fully consistent with the numerical approximation of the objective function. This validates the correctness of the optimization procedure used for training the neural network.

We further demonstrated that the network can perfectly fit the non-linear synthetic datasets (doughnut and squares), while also investigating the minimal network size required to achieve perfect classification. The implementation successfully passed all provided unit tests in 14.430 seconds.

We evaluated the implementation on the Doughnut (Figure 1) and Squares (Figure 2) datasets. The results are summarized in Tables I and II, which show the effect of network capacity, depth, and activation functions on classification performance.

For the Doughnut dataset, increasing the number of neurons significantly improves performance, as the task requires learning a non-linear closed decision boundary. A single neuron is insufficient and yields a linear separator, while three neurons in a single hidden layer provide enough expressive power to approximate the target structure and achieve perfect accuracy.

Deeper architectures such as [2,1] and [1,2] do not outperform a single wider hidden layer. This is due to information bottlenecks introduced by narrow intermediate layers, which restrict the flow of representations and limit the network's ability to form complex decision boundaries. In this task, width is more important than depth.

For the Squares dataset, the decision boundaries are piecewise linear and axis-aligned, which aligns well with the inductive bias of the network. As the number of neurons increases, the model can represent more linear regions, leading to steadily improved performance and perfect accuracy for sufficiently large architectures.

The results in Tables I and II further highlight the impact of activation functions. Linear activations perform poorly as they reduce the network to a single linear transformation. Sigmoid activation consistently achieves the best performance due to its ability to form smooth non-linear boundaries.

ReLU-based networks underperform in smaller architectures, as their piecewise linear nature requires sufficient width to approximate complex shapes, and inactive neurons can limit representational capacity. Mixed activations partially mitigate this but do not outperform fully sigmoid networks.

Overall, the experiments demonstrate that both architectural capacity and activation choice play critical roles in determining the expressive power of neural networks.

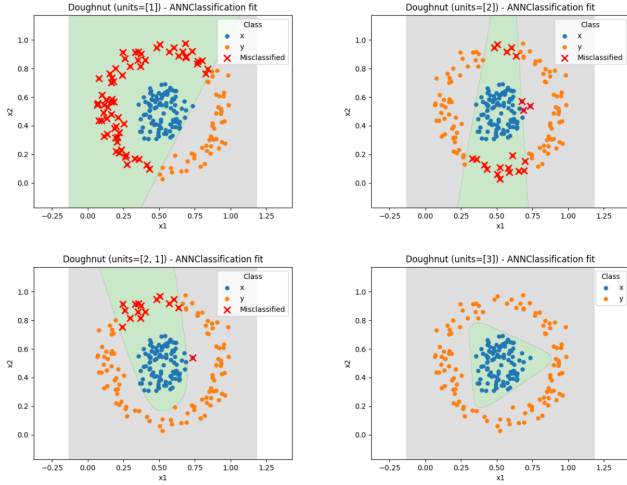


Fig. 1: Doughnut dataset fitted with ANN classifier with different number of neurons and layers

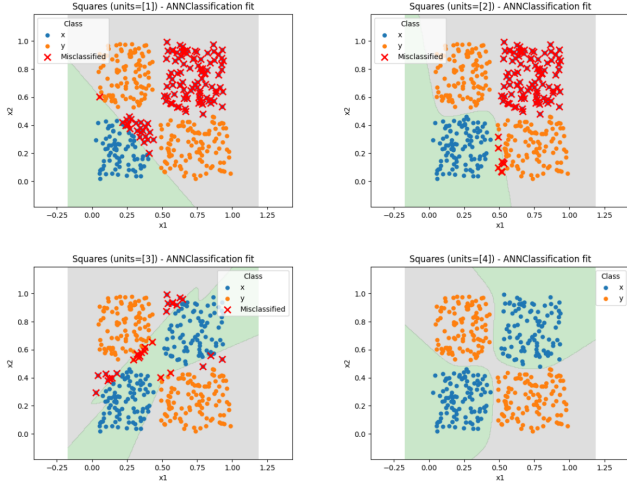


Fig. 2: Square dataset fitted with ANN classifier with different number of neurons.

Units	Sigmoid	ReLU	Linear	Mixed
1	0.699	0.523	0.523	0.523
2	0.903	0.671	0.648	0.685
3	1.000	0.884	0.620	0.898

TABLE I: Accuracy comparison of activation functions on the Doughnut dataset

Units	Sigmoid	ReLU	Linear	Mixed
1	0.703	0.515	0.515	0.705
2	0.743	0.583	0.485	0.685
3	0.925	0.670	0.458	0.893
4	1.000	0.485	0.520	0.515

TABLE II: Accuracy comparison of activation functions on the Squares dataset

TABLE III: Comparison of NumPy and PyTorch implementations (Classification)

Dataset	Architecture	Activation	NumPy Acc	PyTorch Acc
Doughnut	[2]	sigmoid	0.9028	0.9028
		relu	0.6991	0.5231
		linear	0.3843	0.3843
Squares	[3]	sigmoid	1.0000	1.0000
		relu	0.6991	0.7037
		linear	0.3843	0.3843
Squares	[3]	sigmoid	0.8675	0.8675
		relu	0.7050	0.7050
		linear	0.6275	0.6275
Squares	[4]	sigmoid	1.0000	1.0000
		relu	0.7050	0.7050
		linear	0.6275	0.6275

III. PART 2

We implemented both NumPy-based and PyTorch-based versions of the model for classification and regression tasks. The results are summarized in Table III. Overall, the two implementations produce highly consistent results across all datasets, architectures, and activation functions, confirming the correctness of the implementation.

A noticeable discrepancy appears only in the Doughnut dataset for the architecture with two hidden neurons using the ReLU activation function. We attribute this difference to the sensitivity of ReLU networks to small numerical variations during training. In particular, small differences in intermediate activations can lead to neurons switching between active and inactive regimes, which affects gradient propagation and can result in different optimization trajectories between the NumPy and PyTorch implementations.

Despite this isolated case, the overall agreement between both implementations confirms that the forward and backward computations are consistent across frameworks.

IV. PART 3

In this section, we describe the main challenges encountered and the corresponding solutions developed during the final part of the assignment, where the goal was to predict 6 classes from an FTIR spectral dataset of human tissue. Each pixel in the dataset contains spectral information, and the task was to produce a classification map for a test patch of size 200×300 .

The provided baseline model was logistic regression, which achieved a log loss of 0.63734 on the unseen test patch. This served as a reference point for further improvements.

A. Local evaluation

A key challenge in this stage was the limited number of submissions (5 in total), which made reliable local validation essential. To address this, I initially implemented a simple 80/20 train-test split for local evaluation. However, this led to overly optimistic performance estimates. The main issue was that the data was treated as independent

samples, while in reality it has a strong spatial structure. As a result, the training and validation sets contained highly similar neighboring pixels, effectively causing data leakage.

I initially repeated the same mistake in cross-validation, which again resulted in misleadingly high performance. Only later did I redesign the evaluation pipeline to properly reflect the structure of the test set. Specifically, I constructed cross-validation splits based on spatial patches, ensuring that each validation fold corresponded to regions of similar structure and size as the final test patch (of size 200×300).

B. Models

We first explored more complex architectures using only raw spectral data as input. Despite their capacity, both models underperformed, indicating that input representation was more important than model complexity. The CNN achieved a log loss of 0.95962, while the transformer reached 1.18642. These results motivated a shift toward feature engineering and simpler models.

C. Features

Several feature engineering techniques were evaluated. Spectral derivatives were tested to enhance local spectral variations but did not improve performance, suggesting that discriminative information lies in global spectral shape rather than local changes.

Principal Component Analysis (PCA) was then applied for dimensionality reduction, but also failed to improve results, indicating that high-variance directions are not necessarily informative for classification.

In contrast, Standard Normal Variate (SNV) normalization significantly improved performance by removing additive and multiplicative effects and emphasizing spectral shape over magnitude.

To exploit spatial structure, Gaussian smoothing was applied to prediction maps. This reduced noise by enforcing local consistency, leveraging the fact that neighboring pixels are often from the same class. This was the most impactful improvement, highlighting the importance of spatial regularity.

D. Final Solution

The final model incorporates spatially-aware features extracted from local neighborhoods. For each pixel, mean, standard deviation, minimum, and maximum statistics are computed over two neighborhood sizes, capturing both fine and coarse spatial context. These features are concatenated with the original spectral data.

The final feature set is standardized and fed into a simple MLP with one hidden layer (64 ReLU units) and L2 regularization.

Despite its simplicity, this model performs strongly when combined with engineered spatial-spectral features, outperforming more complex deep learning approaches based on raw inputs. Furthermore I implemented deeper neural network with same output which over fitted to the training data (nearly 100% accurate on training data but worse on test data as my simple model).

E. Experiments

We evaluated different parameters of neighborhood features for the best model locally and on the competition set. The results are displayed in the Table IV.

Neighborhood	Validation Accuracy	Test Accuracy
no	0.6002	0.56025
3×3 only	0.5872	0.52066
5×5 only	0.5545	—
3×3 + 5×5	0.5521	0.45193
3×3 + 5×5 + 7×7	0.5482	0.47649

TABLE IV: Performance of neighborhood-based features on validation and test sets. The 5×5 was never tested on the test set only on validation set.

V. CONCLUSION

We identified that naive random splits were inappropriate due to strong spatial correlations in the data, leading to overly optimistic performance estimates. This was resolved by introducing spatially consistent cross-validation based on patch-level splits aligned with the structure of the test set.

The most important improvement came from incorporating spatial context through neighborhood-based features and post-processing. These features enabled a simple MLP to outperform more complex deep learning models by effectively combining spectral and spatial information.

Overall, the results demonstrate that in this setting, careful data representation and spatial awareness are more critical than model complexity. The final solution achieves strong generalization through a combination of robust validation, meaningful feature engineering, and lightweight modeling.